

Coding Lab 6: The Roster That Rewires Itself

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 26, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

6.1 Why this problem

Weeks 3–5 built an expression processor that can tokenize, check brackets, convert, evaluate, and buffer jobs in arrival order. One piece is still missing: evaluating $(a+b)*c$ needs the *values* of a , b , c , and identifiers arrive at unpredictable times in unpredictable numbers. A fixed array demands we guess the capacity up front and pay the slot tax on every front insertion. This lab is that missing piece: a roster of IDs held as a singly linked chain, where each entry carries the address of its successor so the lineup changes by rewriting one address instead of moving n elements. You implement the five operations from lecture — insertion (front, end, position), deletion, search, display, and in-place reverse — with the same pointer discipline the deck drilled: save the only path *before* you overwrite it, and free exactly once what you malloc'd.

6.2 Your task

Implement the singly linked list ADT in C on heap nodes with explicit contracts on every fallible operation. The contract is given to you in `slist.h` — *do not modify it*. Copy `starter_slist.c` to `slist.c` and fill in all nine functions:

```
SNode *slist_insert_front(SNode **head, int x);
SNode *slist_insert_end(SNode **head, int x);
int     slist_insert_at(SNode **head, size_t pos, int x); /* 0 ok / -1 fail */
int     slist_delete_value(SNode **head, int x);          /* 0 gone / -1 absent */
SNode *slist_search(SNode *head, int x);                 /* NULL when absent */
size_t  slist_length(const SNode *head);
int     slist_display(const SNode *head, int *out, int out_cap);
void     slist_reverse(SNode **head);
void     slist_free(SNode **head);
```

Insertion is the deck's walk-then-splice. Front-insert is the two-write sequence (`newNode->next = *head` first, *then* `*head = newNode`) — swapped, the old chain is orphaned with no copy anywhere. End-insert walks to the last node (an empty list sets `*head` directly); position insert walks `p` to the predecessor then splices in the same order, refusing `pos` past the end. Every `malloc` is checked for NULL before dereferencing.

Deletion is bypass, reclaim, never touch. Delete the *first* node holding `x` (head first, else the predecessor walk): rewrite one link, **free** exactly once, never touch the node afterwards. Absent `x` changes nothing and reports `-1`; only the first duplicate goes.

Reverse is the three-pointer loop. `prev = NULL, curr = *head`; save `next`, flip, advance both walkers; `*head = prev`. Display never disturbs the list (the traversal pointer moves while `head` stays put), and **free** releases every node and NULLs `*head` so a second free is safe.

6.3 Constraints

- Representation must be the heap-chained `SNode` nodes of `slist.h` — no arrays, no global/static mutable state.
- Every allocation is checked (`malloc` for `NULL`); every node is freed exactly once; no use-after-free, no writes past allocated bounds. Error paths leave the list unchanged.
- Correctness is graded against the contract; your code must compile with `clang -std=c11 -Wall -Wextra -Werror` without errors or warnings.
- Complexity: front-insert $O(1)$ worst case; end/position insert, delete, and search $O(n)$ worst case in the current length (search $O(1)$ best case when `head` holds `x`); display $\Theta(n)$; reverse $O(n)$ worst case in $O(1)$ auxiliary space; length and free $O(n)$ worst case.

6.4 Worked examples (these are the visible tests)

Example 1 — build the roster. Front-insert 42 (chain: 42), front-insert 17 (chain: 17 $\rightarrow$ 42), end-insert 88 (chain: 17 $\rightarrow$ 42 $\rightarrow$ 88). Length is 3; display writes 17, 42, 88. Freeing sets `head` to `NULL`.

Example 2 — splice, delete, search. From 17 $\rightarrow$ 42 $\rightarrow$ 88: insert 63 at position 1 giving 17 $\rightarrow$ 63 $\rightarrow$ 42 $\rightarrow$ 88; delete the first 42 giving 17 $\rightarrow$ 63 $\rightarrow$ 88 with length 3; `search(63)` finds its node while `search(77)` reports absent (`NULL`).

Example 3 — reverse and edges. Reversing 17 $\rightarrow$ 63 $\rightarrow$ 88 gives 88 $\rightarrow$ 63 $\rightarrow$ 17; deleting the head 88 leaves length 2; reversing again gives 17 $\rightarrow$ 63. Reversing the freed (empty) list is a safe no-op with length 0.

6.5 What to submit

1. `slist.c` — your completed implementation (edit `starter_slist.c`).
2. `slist.h` — unchanged, but include it so the submission compiles standalone.

6.6 Getting started

1. Copy `starter_slist.c` to `slist.c` and fill in the nine function bodies.
2. Sanity-check locally:

```
gcc -std=c11 -Wall -Wextra -o test_visible test_visible.c slist.c
./test_visible
```

All three worked examples should pass.

3. Submit `slist.c` and `slist.h`. The autograder compiles your `slist.c` against a *hidden* test suite and reports pass/fail. The hidden suite covers: NULL head-pointer safety for every op; front/end/position insert boundaries including `pos == length` vs. past-the-end refusal with the list unchanged; head/middle/tail/absent/empty deletes with first-occurrence-only duplicates; search identity at head and tail; display validation with too-small, NULL, and negative-capacity buffers plus a canary check that nothing is written on refusal; exact-capacity display and head purity; INT_MIN/INT_MAX/-1/0 round-trips; reverse of empty/single/two/four chains with double-reverse identity; free with head-NULLing and double-free safety; and a seeded 200-operation random campaign checked operation-by-operation against an independent array oracle.